



3. Filter Function -

- Filter Function in a python is a useful function that applies a given function to each item of a iterable (list, set, tuple) & we filter the items on the basis of if condition.

Eg.

Map Function

```
iterable ← numbers = [1, 2, 3, 4, 5]
def square(x):
    return x**2 } ← function
result = map(square, numbers)
print(list(result))
> [1, 4, 9, 16, 25]
```

Filter Function

Syntax :

Filter (function, iterable)

- Filter function returns true or false for elements where condition is true, those elements are available in final output

Eg. Filter out odd elements & keep even elements using Filter.

```
iterable ← numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
def is_even(x):
    return x % 2 == 0 } → function
```



```
result = Filter(is-even, numbers)
print(list(result))
> [2, 4, 6, 8, 10]
```

Using lambda Function

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even-num = filter(lambda x: x % 2 == 0, numbers)
result = list(even-numbers)
print(result)
```

Eg Q2. using filter function create a new list where elements are less than equal to 5

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
def equal(x):
    return x <= 5
result = filter(equal, numbers)
print(list(result))
> [1, 2, 3, 4, 5]
```

Using lambda function -

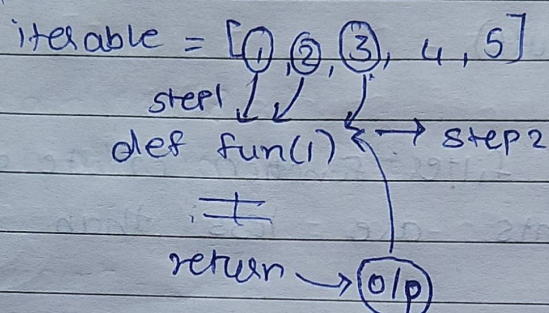
```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
equal = filter(lambda x: x <= 5, numbers)
result = list(equal)
print(result)
```



4. Reduce Function -

- Reduce function takes first two items of iterable (list, tuple, set, range, str) & applies to function, next we take o/p of function & next item in the iterable & ^{applies} function to func & we do this till the end of iterable & at end we get single cumulative value.

How it works :



[^{from} import functools import reduce]

Syntax :

reduce(function, iterable)

Eg.

numbers = [1, 2, 3, 4, 5]

def add(x, y):

return x + y

result = reduce(add, numbers)

> 15

using lambda

numbers = [1, 2, 3, 4, 5]

add = reduce(lambda x, y: x + y, numbers)

print(add)



Exception Handling : While working on ~~also~~ Python we get two types of Errors - 1) Syntax Error 2) Exception (Runtime error)

1. Arithmetic errors -

i) Zero Division Error -

When any number is divided by zero

Eg. $10/0$

```
num1 = int(input("Enter first number :"))
num2 = int(input("Enter second number :"))
```

```
result = num1 / num2
>Error
```

Exception Handling Syntax :

try :

← risky code

(exception/run time errors)

except :

← will contain that

Eg.

```
try :
```

```
    result = num1 / num2
```

```
    print(result)
```

```
except ZeroDivisionError as err:
```

```
    print(err)
```

2] overflow error -

int $\rightarrow$ No limit

Float $\rightarrow 1.8e^{308}$

Overflow error raised when a floating point arithmetic result is too large to be represented.

Eg.

```
math.exp(800)
> Math range Error
```

```
import math
```

```
try :
```

```
    value = math.exp(800)
```

```
    print(value)
```

```
except OverflowError as err :
```

```
    print(err)
```

2. Type & Value Error -

1] Type Error -

Type Error raised when an operation is applied to an object of an inappropriate type.

Eg. ~~int~~ 5 + int

"hello" + 10

> Type Error

2
Date ____ / ____ / ____

```
try :  
    "hello" + 10  
except TypeError as err :  
    print(err)
```